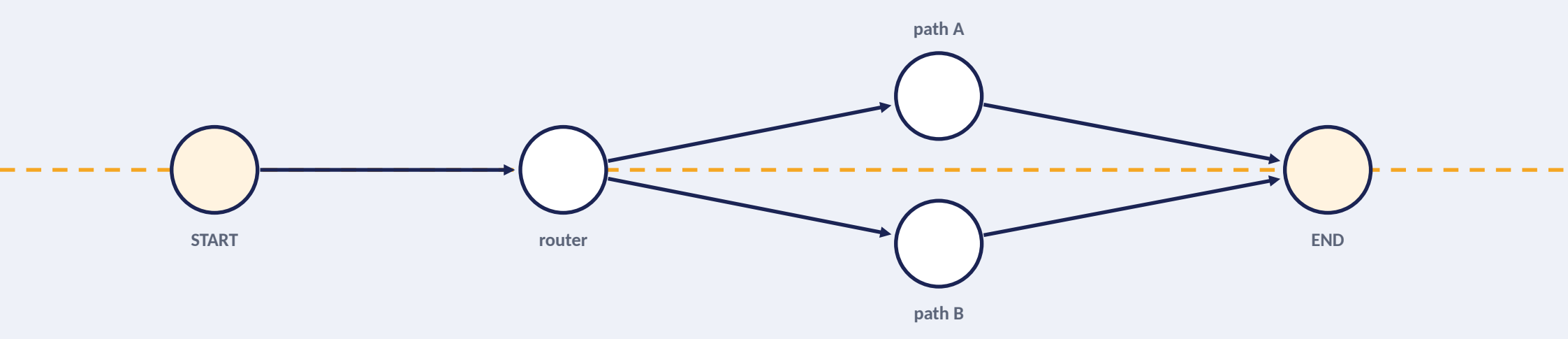
LANGGRAPH · CHAPTER 4 · FLASHCARDS

Conditional Edges

Graphs that decide

10 code cards on routers, mappings and loops that stop



Nodes write the evidence

Worker nodes return partial updates; none of them chooses the next step

- grade_node only writes score into the state; the decision comes later, from a router.
- With the score in state you can log it, test the threshold and reuse it in a second decision.

PYTHON

```python
class AgentState(TypedDict):
    answer: str
    score: int        # the evidence the router will read
    result: str


def grade_node(state: AgentState) -> dict:
    # Fake grading: mention "RAG" and you pass (no model, no API key)
    score = 90 if "rag" in state["answer"].lower() else 50
    return {"score": score}


def pass_node(state: AgentState) -> dict:
    return {"result": "Passed"}


def retry_node(state: AgentState) -> dict:
    return {"result": "Retry needed"}
```

Reads state, returns a name

A router is a plain function of the state that returns the next node's name as a string

- A node returns a dict and may change state; a router returns a string and only reads.
- In the diagram a node is a box; a router is the set of branching arrows itself.

PYTHON

```python
def route_after_grade(state: AgentState) -> str:
    if state["score"] >= 70:
        return "pass_node"
    return "retry_node"

#                 node                          router
# registered by   add_node                      add_conditional_edges
# returns         a dict (a state update)       a str (the next node's name)
# changes state?  yes                           no: it only reads
```

Source, router, mapping

After the source node runs, LangGraph calls the router and follows the mapped name

- The mapping lists every possible destination, so compile() can check them all.
- It also lets the drawing show both dotted branches without running the graph.

PYTHON

```python
graph = StateGraph(AgentState)
graph.add_node("grade_node", grade_node)
graph.add_node("pass_node", pass_node)
graph.add_node("retry_node", retry_node)
graph.add_edge(START, "grade_node")

graph.add_conditional_edges(
    "grade_node",            # after THIS node runs...
    route_after_grade,       # ...call this router with the state...
    {"pass_node": "pass_node", "retry_node": "retry_node"},  # ...map its answer
)

graph.add_edge("pass_node", END)     # give each branch an exit
graph.add_edge("retry_node", END)
app = graph.compile()
```

Labels decouple logic from wiring

The router judges quality; the mapping alone decides which node each label means

- Repointing "bad" from retry_node to review_node changes one dictionary entry, not the router.
- Prefer abstract labels once a graph has more than a couple of branches.

PYTHON

```python
def route_after_grade(state: AgentState) -> str:
    # Knows about quality, not about any node's name
    return "good" if state["score"] >= 70 else "bad"


def build(mapping: dict[str, str]):
    graph = StateGraph(AgentState)
    ...                             # nodes, START edge, each node's edge to END
    graph.add_conditional_edges("grade_node", route_after_grade, mapping)
    return graph.compile()


weak_answer = {"answer": "I am not sure.", "score": 0, "result": ""}
build({"good": "pass_node", "bad": "retry_node"}).invoke(weak_answer)
# result: 'Retry needed'
build({"good": "pass_node", "bad": "review_node"}).invoke(weak_answer)
# result: 'Sent to a human reviewer'
```

Who writes, who reads

The node writes the evidence, the router reads it, the chosen branch writes the result

- The router runs between two nodes and changes nothing in the state.
- The wiring is fixed; which path runs through it is decided per invocation.

TEXT

```
{"answer": "RAG means...", "score": 0,  "result": ""}
    |  grade_node finds "rag", returns {"score": 90}
    v
{"answer": "RAG means...", "score": 90, "result": ""}
    |  route_after_grade reads score: 90 >= 70 -> "pass_node"
    |  (no state change: routers never write)
    v
{"answer": "RAG means...", "score": 90, "result": "Passed"}
    |  pass_node -> END; retry_node never runs

"I am not sure." -> score 50 -> "retry_node" -> result "Retry needed"
```

Unknown names: error or silence

With a mapping an unknown label raises; without one it is ignored and the run just stops

- A mapping entry pointing at a missing node is caught even earlier, at compile().
- Without a mapping, a typo in the router only logs a warning: result stays empty.

PYTHON

```python
def route_typo(state: AgentState) -> str:
    return "pass_nod"                          # typo

# 1. With a mapping: the run fails loudly
mapping = {"pass_node": "pass_node", "retry_node": "retry_node"}
graph.add_conditional_edges("grade_node", route_typo, mapping)
# KeyError: 'pass_nod'

# 2. Without a mapping: a log line, then the run simply ends
graph.add_conditional_edges("grade_node", route_typo)
# ... wrote to unknown channel branch:to:pass_nod, ignoring it.
# {'answer': 'RAG', 'score': 90, 'result': ''}

# 3. A mapping entry naming a node never added: compile() refuses it
#    {"bad": "review_node"}  ->  ValueError: ... unknown target 'review_node'

# Checked on langgraph 1.2.6 (this snippet is ours)
```

A conditional edge that points backward

Send retry back to the grader, and give the router an exit that does not need a pass

- Some node must increment attempts, or the counted exit never fires.
- This is the skeleton of Chapter 5's evaluator-optimizer and Chapter 7's agent loop.

PYTHON

```python
class AgentState(TypedDict):
    ...                    # answer, score, result as before
    attempts: int          # NEW: how many grading passes so far


def grade_node(state: AgentState) -> dict:      # count every pass here
    score = 90 if "rag" in state["answer"].lower() else 50
    return {"score": score, "attempts": state["attempts"] + 1}


def route_after_grade(state: AgentState) -> str:
    if state["score"] >= 70:
        return "pass_node"          # good enough -> exit
    if state["attempts"] >= 3:
        return "pass_node"          # give up after 3 tries -> exit anyway
    return "retry_node"             # otherwise loop back

graph.add_edge("retry_node", "grade_node")      # the back edge
# a weak answer runs: grade, retry, grade, retry, grade, pass_node
```

The backstop behind your exit

LangGraph stops a run after recursion_limit super-steps with GraphRecursionError

- The default is 10,007 steps on langgraph 1.2.6 (older releases used 25).
- A looping LLM call could make thousands of paid calls first: set the limit on every loop.

PYTHON

```python
from langgraph.errors import GraphRecursionError

# Loop with NO counted exit, and a grader that always returns 40
try:
    app.invoke(initial_state, config={"recursion_limit": 10})
except GraphRecursionError as err:
    print(err)
# Recursion limit of 10 reached without hitting a stop condition. ...
# 10 steps ran first: five laps of grade + retry

# For a loop that works, keep a sane ceiling anyway:
app.invoke(initial_state, config={"recursion_limit": 50})

# A safety net, not a strategy: the crash says the loop is broken
```

PITFALL · GIVING UP

Giving up is not passing

Listing 4.5 sends the give-up case to pass_node, so a failed answer ends as "Passed"

- Route the give-up exit to its own node so the final state tells the truth.
- The router stays plain: one more label and one more mapping entry.

PYTHON

```python
# As in Listing 4.5: three failed tries end in pass_node
# {'answer': '...', 'score': 50, 'attempts': 3, 'result': 'Passed'}

def give_up_node(state: AgentState) -> dict:
    return {"result": f"Gave up after {state['attempts']} tries"}


def route_after_grade(state: AgentState) -> str:
    if state["score"] >= 70:
        return "pass_node"
    if state["attempts"] >= 3:
        return "give_up_node"       # an honest exit
    return "retry_node"

graph.add_node("give_up_node", give_up_node)
graph.add_edge("give_up_node", END)
# a fix of ours, not the book's code
```

Quiet branching mistakes

A branch with no exit still compiles, and a router's writes are thrown away

- A node with no outgoing edge ends the run after it: compile() does not flag it.
- A router that edits state changes nothing; write evidence in a node, then route on it.

PYTHON

```python
graph.add_edge("pass_node", END)
# forgot: graph.add_edge("retry_node", END)
app = graph.compile()                    # no error
app.invoke({"answer": "no", "score": 0, "result": ""})
# {'answer': 'no', 'score': 50, 'result': 'Retry needed'}  <- run stopped there

# (state now has an extra field: note: str)
def route_after_grade(state: AgentState) -> str:
    state["note"] = "set by router"      # mutating state in a router...
    return "pass_node" if state["score"] >= 70 else "retry_node"
# final state: note is still ''          # ...is silently discarded

# Checked on langgraph 1.2.6
```